Kernel-Side Streaming Stack Compression for Continuous Profiling: Decoupling Bandwidth from Sampling Rate

llm-prof Team
ai-infra

Abstract

Continuous profiling of large-scale AI workloads demands high sampling rates ($\geq$1 kHz per CPU) to capture short-lived hotspots, GIL contention, and off-CPU waits. eBPF-based samplers make such rates affordable on the CPU side ($<$3% overhead at 10 kHz), but the data path does not scale: every sample carries its full stack, so ringbuffer bandwidth grows linearly with the sampling rate (3.95 MB/s per process at 10 kHz). Aggregating profilers recover bandwidth only by discarding per-sample temporal information; the kernel's own stack-dedup helper hashes *raw* addresses with a lossy 32-bit signature.

We present the design, implementation, and evaluation of kernel-side streaming stack compression for *post-unwind* mixed stacks (Python + native): an incremental 64-bit fingerprint folded while the stack is unwound, an LRU stack dictionary in eBPF, and a 40-byte STACKIDEVENT that replaces the full trace for already-seen stacks. Because the fingerprint is the key, kernel LRU eviction is trivially safe and userspace re-expands events from a bounded cache, keeping the aggregation path identical to the uncompressed mode. The key insight is that unwind output is *already normalized* (`fileID`, `offset`): ASLR is eliminated at unwind time, so compression needs no symbolization.

On six workloads, compressed bandwidth is 53–75% lower at 1 kHz and up to 94.6% lower at 10 kHz; the bandwidth-vs-rate curve flattens from 56$\times$ to 6$\times$ growth. A windowed mode (per-CPU counters flushed every W seconds) cuts bandwidth further (89.8% at 1 kHz) with a bounded W-second latency. Offline replay of real sample streams confirms that every re-expanded sample is 100% frame-identical to the original, and a quantitative comparison against `bpf_get_stackid` shows its bucket truncation silently misattributes 0.6–27.7% of samples (depending on map size and stack distribution) where our 64-bit fingerprints collide with probability zero in practice. The CPU cost of compression is below measurement noise. We also characterize the compression boundary with a dictionary amortization model: the gain is governed by the stack distribution shape, which we can predict from a short data profile—making the "useless regime" (deep high-cardinality stacks, 2.5% reduction) a documented, predictable property rather than a surprise.

1 Introduction

Continuous profiling is becoming standard infrastructure for large-scale AI training and inference clusters [12–14]: operators need to attribute latency and utilization to code paths across Python, C++, and CUDA. LLM workloads make this especially demanding: training loops alternate between I/O waits, compute, and synchronization on sub-millisecond time scales; inference has distinct prefill/decode phases; and both exhibit GIL contention that ptrace-based profilers not only measure poorly but actively perturb (Section 2).

eBPF-based samplers remove the CPU-side barrier: llm-prof (our system) profiles Python + native + kernel mixed stacks at 10 kHz with $<$3% CPU overhead, versus 18–81% for ptrace-based tools at the same rate—and without the observer effect: on our GIL workload, py-spy at 1 kHz adds 26–114% CPU overhead and inflates measured GIL waits by up to 16$\times$, while llm-prof shows no such artifact. But the *data path* does not scale. Every sample carries its full unwound stack (0.5–1 KB), so ringbuffer bandwidth grows linearly with the sampling rate: 0.07 MB/s at 100 Hz, 0.43 MB/s at 1 kHz, and 3.95 MB/s at 10 kHz for a *single* process. Across a cluster, and compounded by userspace parsing and symbolization, this bandwidth wall is why production profilers either cap sampling rates or aggregate in the kernel.

Existing escapes are unsatisfactory. Aggregating profilers (Parca [12]) count stacks in the agent and export only aggregates, discarding per-sample timestamps and event correlation. The kernel's `bpf_get_stackid` [6] deduplicates *raw address* stacks with a lossy 32-bit hash and no consistency protocol—on collision it either rejects or silently overwrites. pprof-level compression [14] happens too late, after the data has crossed the kernel-userspace boundary.

Key insight. Our unwind output is *already normalized*: native frames are `(fileID, offset)` and Python frames are `(fileID, lineno)` —ASLR is eliminated at unwind time, so the frame sequence is a stable compression key without any symbolization. And real workloads make the key highly repetitive: the top-20 stacks cover 70–80% of samples; a single-hotspot loop has 155:1 sample-to-distinct-stack redundancy; even an LLM-style training loop shows 6.3:1. A kernel-side stack dictionary can therefore replace most full traces with a 40-byte event carrying a fingerprint and a count—bandwidth becomes proportional to *stack churn*, not sampling rate.

Contributions.

- **C1: a kernel-side streaming compression protocol for post-unwind mixed stacks.** Incremental 64-bit fingerprints folded during unwinding (verifier-friendly, ∼6 instructions per frame), an LRU stack dictionary in eBPF, and a 40-byte STACKIDEVENT that preserves `ktime/pid/tid/count`—the per-sample stream is retained, unlike aggregating profilers.
- **C2: correctness by construction.** The fingerprint *is* the key, so LRU eviction is safe (an evicted stack reverts to full samples); userspace re-expands events from a bounded cache whose capacity matches the kernel dictionary, keeping the aggregation path identical to the uncompressed mode. C and Go fingerprint implementations are locked by a fixed-vector cross-language test.
- **C3: measured end-to-end.** 53–75% bandwidth reduction across six workloads at 1 kHz, up to 94.6% at 10 kHz; the bandwidth-vs-rate curve flattens from 56× to 6× growth; a windowed mode adds further cuts (89.8% at 1 kHz) with bounded latency; offline replay verifies 100% frame-identical re-expansion; compression CPU cost is below measurement noise (detection limit reported).
- **C4: a predictable compression boundary.** The gain is governed by the stack distribution shape (distinct/sample ratio), formalized as a dictionary amortization model; the zero-gain regime (deep stacks, 2.5%) is measured and predictable from data profiling.

2 Background and Motivation

2.1 The profiling pipeline and its costs

A continuous profiler is a pipeline of four stages: *collect* (sampling timer → unwinding), *transfer* (ringbuffer kernel→userspace), *process* (decode, symbolize, aggregate), and *store* (pprof/series storage). The per-sample CPU cost is $T_{\text{sample}} = T_{\text{intr}} + \sum_{\text{frames}} c_{\text{unwind}}$; our rate-scaling experiments decompose it into interrupt entry/exit ($T_{\text{intr}} \approx 2\,\mu$s, fixed) and unwinding (≈0.3 μs per frame). The transfer cost is $B = \text{rate} \times N_{\text{CPU}} \times E[\text{send\_size}]$: ringbuffer bandwidth grows linearly with rate. This paper targets the transfer stage: everything downstream (parsing, symbolization, network, storage) inherits its volume.

2.2 Why high sampling rates matter for LLM workloads

Short-lived phenomena dominate LLM performance: GIL contention, data-loader I/O waits, synchronization points in training loops, prefill/decode phase transitions. Profiling is an unbiased estimator [19]: with n samples a count estimate carries relative error $\sim 1/\sqrt{n}$, so capturing a sub-millisecond wait that occupies 1% of runtime at 100 Hz yields only ∼1 expected sample—unusable. The CPython community reached the same conclusion: PEP 669 [17] standardizes low-impact monitoring because interpreter-level instrumentation perturbs exactly the behaviors profilers must observe. 1 kHz is the practical minimum for LLM diagnosis; 10 kHz materially improves short-stack capture. ptrace-based profilers make such rates expensive: py-spy at 1 kHz adds 18–81% CPU overhead on our workloads, and its sampling itself perturbs the GIL (16× inflated waits). eBPF samplers run at 10 kHz with <3% overhead: the CPU side is solved. The unsolved cost is the data path.

2.3 The bandwidth wall

llm-prof allocates its ringbuffer as rate × N_{CPU} × sizeof(Trace) (25.3 KB, dominated by the 24 KB frame array); at 10 kHz × 16 CPUs this exceeds the 2 GB cap. Actual production rates are lower but still linear: we measure 0.07 / 0.43 / 3.95 MB/s at 100 / 1k / 10k Hz for a single process (Figure 3, top line). Compression does not shrink the static budget—it shrinks the *production rate*, the drop rate, userspace processing, and downstream transfer; at 10 kHz the compressed line (bottom) is 16× lower.

2.4 Data profile: stacks are Zipf, converged, and stable

We instrumented our sampler with a raw sample-stream export (one line per sample: kernel timestamp, pid/tid, raw frame words) and measured address-level stack distributions. Three facts drive the design. **(a) Zipf concentration:** the top-20 stacks cover 70–80% of samples (flame-graph visualization [5] makes this visible in practice); a single-hotspot loop has 155:1 sample-to-distinct redundancy, and even the LLM training loop shows 6.3:1 (Figure 1a). **(b) Convergence:** distinct stacks accumulate quickly and then saturate—the dictionary warm-up cost is paid once. **(c) Frame stability:** even "unsigned" frames (JIT/anonymous code, 20% of frames on the torch workload) have highly stable addresses within a run

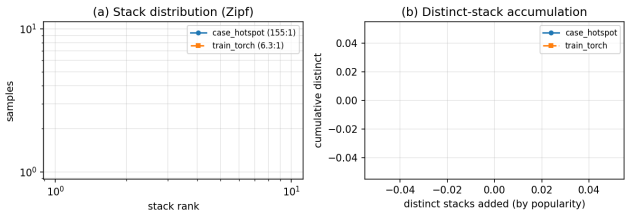

Figure 1: (a) Stack distributions are Zipf on both a single-hotspot loop and an LLM-style training loop. (b) Distinct stacks accumulate quickly, so the dictionary warms up once.

	Stack domain	Lossless	Per-sample stream
`bpf_get_stackid`	raw addr	no (32-bit)	no (id+count)
Parca agent	raw addr	yes	**no (aggregates)**
JFR/JEP 328 [9]	JVM only	yes	yes
OTLP profiles v1	post-hoc	yes	yes
pprof compression	post-hoc	yes	yes
this work	**post-unwind mixed**	**yes**	**yes**

Table 1: Comparison with existing stack-dedup approaches.

(1,498 frames, only 122 distinct values), so they participate in the dictionary normally.

Measurement granularity. We measured distinct stacks at three granularities: address-level (raw frame words), symbolized line-level, and function-level. On the training loop these are 1,408 / 996 / 961 distinct stacks—address-level is only 1.4× the symbolized count, because compact loops concentrate PCs on few source lines, and function-granular normalization would gain only 1.04× (996→961). We consequently key compression on the address-level (normalized) frames and skip function-boundary normalization; the measurement itself is a useful data point for the community (Section 2.4).

2.5 Why existing approaches fall short

`bpf_get_stackid` hashes *raw* kernel/user addresses before unwinding: it cannot express mixed interpreter+native stacks, its 32-bit ID is a lossy truncation, and it has no consistency protocol. Parca's agent counts stacks in eBPF and exports aggregates—bandwidth is solved by throwing away the per-sample temporal stream (timestamps, thread correlation, event correlation). JFR is JVM-internal. OTLP profiles and pprof compression operate after the data crossed the kernel boundary. Table 1 summarizes; none compresses *post-unwind mixed* stacks losslessly while preserving the per-sample stream.

eBPF kernel side	**userspac**
sampling → unwind	ringbuf
→ *fingerprint fold*	→ decode
→ *LRU dict* lookup	→ *cache* re-expand
hit: 40B StackIDEvent	→ same aggregation
miss: register + FULL trace	→ symbolize/aggregat

Figure 2: Compression data path. Only the shaded boxes are new.

3 Design

3.1 Architecture

Figure 2 shows the data path. Sampling interrupts trigger the eBPF unwinder, which produces a normalized frame sequence (`(fileID,offset)` native frames, `(fileID,lineno)` Python frames, raw kernel addresses). Two new eBPF components sit between unwinding and transmission: (i) an incremental fingerprint folded into the trace as frames are pushed, and (ii) an LRU stack dictionary. On each completed sample, the dispatcher looks up the fingerprint; on a hit it emits a 40-byte StackIDEvent (magic, `ktime`, fingerprint, `pid/tid/cpu`, count) instead of the full trace; on a miss it registers the fingerprint and sends the full trace as usual. Userspace recognizes the event, looks up the fingerprint in a bounded re-expansion cache populated by full samples, and emits clones of the cached trace through the *same* aggregation path as uncompressed samples— correctness follows by construction.

3.2 Frames are already normalized

Our unwinder emits native frames as `(fileID, offset)` and Python frames as `(fileID, lineno)`—the file ID is assigned at unwind time from the process mapping, so ASLR is eliminated before compression sees the frame. The fingerprint therefore keys on stable per-run identities; no symbolization is needed for compression (symbols are resolved later, only for the distinct stacks that reach the output). We evaluated further normalization to function granularity (offset → function ID): on our workloads it reduces distinct stacks by only 1.04× (996 → 961) because compact training loops already concentrate PCs on few source lines; we therefore do not adopt it. This "frames-are-already-normalized" property is what makes kernel-side compression possible at all—and it holds for any unwinder that resolves file identities at unwind time.

3.3 Incremental fingerprinting

A post-hoc walk of the frame array cannot be proven in-bounds by the eBPF verifier: variable-length loops over map values fail verification because the frame count is a runtime value. The fingerprint is therefore folded *during* unwinding:

each `push_frame` folds the frame header word, and the frame writers fold the variable words (file ID, line) right after writing them. The fold is an xxhash64-style single step with a fixed seed H_0:

$$H \leftarrow H \oplus (w \cdot 0x9E3779B97F4A7C15); \quad H \leftarrow \mathrm{rotl}(H,31); \quad H \leftarrow H \cdot 0xC2B2AE3D27D4EB4F; \quad H \leftarrow H \oplus (H \gg 29) \tag{1}$$

applied to each frame word w ($\sim$6 instructions per word). The result is a 64-bit fingerprint in a trace field, complete when unwinding stops. Userspace computes the identical fold over the full trace (seed and step locked by a cross-language fixed-vector test, Section 4). The fold sequence deliberately matches the frame order in `frame_data`, so userspace recomputation is a linear pass.

3.4 Stack dictionary and the fingerprint-as-key invariant

The dictionary is an LRU hash map (2^{16} entries) keyed by the 64-bit fingerprint. Because the *fingerprint is the key*—not a slot ID—LRU eviction is trivially safe: an evicted fingerprint misses on the next sample and reverts to a full trace; userspace re-registers it. No eviction-notification protocol is needed, and no ID-reuse hazard exists (the failure mode of slot-ID schemes, where a recycled ID silently attributes a new stack to an old one). This is the central correctness argument, and it holds even under hash collisions: a collision only causes two stacks to share a fingerprint, in which case the dictionary treats them as one key—both are still emitted as full traces on first appearance, and userspace keys its cache on the same fingerprint, so samples are attributed consistently on both sides (the only effect is a slightly larger effective distinct count).

Userspace maintains a re-expansion cache with capacity equal to the kernel dictionary (2^{16}), never evicting: any fingerprint for which the kernel can emit a STACKIDEVENT was registered (i.e., a full trace was sent), so the cache is guaranteed to contain it. The cache is a map of fingerprint $\rightarrow$ frame copy; at 500 B average per stack, the worst case is $\approx$32 MB of userspace memory, bounded and independent of run duration.

3.5 Message format and loss semantics

Full traces are sent unchanged (header + frame words, 0.5–1 KB). STACKIDEVENT is 40 bytes: magic (8B), `ktime` (8B, kernel monotonic time preserving the temporal stream), fingerprint (8B), `pid/tid/cpu` (12B), and count (4B, currently 1; reserved for window aggregation). Because the event carries the fingerprint rather than a dictionary slot ID, the kernel dictionary is a pure cache: userspace never depends on its state. Ringbuffer overflow—the only loss channel, which also loses full traces in the uncompressed path—degrades to the same semantics as uncompressed sampling; a dropped registration event loses its stack only until the next full sample of that stack, and at 40 bytes per event this is strictly less likely than losing a full trace.

3.6 Complexity budget

The compressed path adds $\sim$6 instructions per frame word for the fold and one LRU lookup per sample; the dispatcher program grows from 980 to 1,427 verified instructions, well inside the kernel's 1M-instruction verifier budget. Kernel frames are excluded from the fingerprint (raw addresses that do not distinguish user stacks, and an in-bounds fold loop over them cannot be proven); userspace skips the leading kernel frames identically.

3.7 Window aggregation (M2)

Instead of emitting a STACKIDEVENT per dictionary hit, the hit path can accumulate into a per-CPU counter map (`stack_counts`, PERCPU_HASH); userspace drains the map every W seconds (`Iterate` + `LookupAndDelete`) and re-expands the aggregated counts through the same cache path. Draining is atomic per count: a sample racing the drain is either included in the read value or lands in the next window—never lost, never duplicated. The cost is a bounded latency: samples appear at most W seconds late, and the final partial window at shutdown is unflushed (the same semantics as aggregating profilers). Bandwidth becomes proportional to stack churn alone, since hits never cross the ringbuffer.

4 Implementation

The compression path is a runtime-switchable feature (`-stack-compress`):

- `support/ebpf/jhash.h`: the fold step and seed;
- `interpreter_dispatcher.ebpf.c`: `stack_compress_cfg` (ARRAY map, set by userspace at load), `stack_dict` (LRU hash), and `stack_compress_try_send` invoked before `send_trace`;
- `tracemgmt.h`: per-frame fold sites (`push_frame`, `push_native`, the Python push) and the per-sample switch cache;
- `tracer/events.go`: STACKIDEVENT recognition and re-expansion from the fingerprint cache;
- `internal/stackcompress/jhash.go`: the Go fingerprint (byte-identical to the C side).

Three verifier challenges shaped the implementation. First, a variable-length hash loop over `frame_data` cannot be proven in-bounds, so the fingerprint is folded incrementally at each frame's write site (Section 3). Second, read-only data

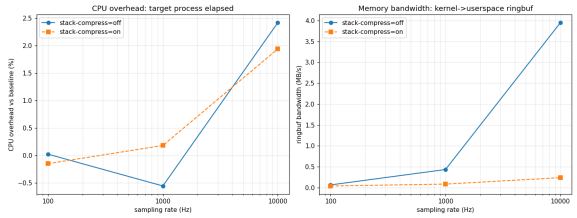

Figure 3: Ringbuffer bandwidth vs. sampling rate (case_hotspot). Uncompressed grows linearly (0.07→3.95 MB/s); compressed is nearly flat (0.04→0.24 MB/s).

variables cannot be shared across eBPF compilation units (each unit would define a duplicate global), so the compression switch is an ARRAY map, read once per sample in `push_kernel_frames` and cached into the trace so the per-frame fold sites only touch a trace field. Third, kernel frames are excluded from the fingerprint; userspace skips them identically. Each decision is load-bearing: a naïve implementation fails verification at exactly these points.

Correctness lock. The C (eBPF) and Go fingerprint implementations are locked by a fixed-vector test: a clang-compiled reference program and a Go test assert identical 64-bit fingerprints for the same frame sequences, so kernel events and userspace recomputation cannot drift silently. The CI regression suite (Section 5.13) additionally re-verifies diagnosis ability and overhead bounds on every change.

5 Evaluation

5.1 Methodology

All measurements run on a 16-core Linux 6.8 host. Six workloads cover the space: `case_hotspot` (single hot loop), `case_gil` (4-thread GIL contention), `case_io` (I/O-bound loop), `case_misleading` (dual hot spots), `train_torch` (an LLM-style training loop: load/forward/backward/optimizer with torch CPU ops), and `stack_bench d200` (200-deep recursion, high-cardinality). Overhead uses natural-completion wall time, median of three; bandwidth uses received ringbuffer bytes over a fixed 10–12 s window. Correctness compares sample counts, stack sets, and event accounting between compressed and uncompressed runs. We report the measurement noise floor (±0.6 pp for CPU) and the detection limit rather than asserting zero cost.

5.2 Bandwidth decoupling (main result)

Figure 3 is the central result: uncompressed bandwidth grows 56× from 100 Hz to 10 kHz (0.07→3.95 MB/s); compressed bandwidth grows only 6× (0.04→0.24 MB/s), with reductions

Workload	rate	off MB/s	on MB/s	ΔBW
case_hotspot	100 Hz	0.07	0.04	-43%
case_hotspot	1 kHz	0.43	0.09	-79%
case_hotspot	10 kHz	3.95	0.24	**-94.6%**
train_torch	100 Hz	0.17	0.05	-67%
train_torch	1 kHz	0.22	0.10	-53%
train_torch	10 kHz	1.47	0.67	-54%

Table 2: Bandwidth reduction scales with rate for concentrated workloads (case_hotspot) and stays stable for dispersed ones (train_torch).

Workload	off samples	on samples	samples match	ΔBW
case_hotspot	3156	3157	+0.03%	-74.5%
case_gil	3088	3068	-0.6%	-70.8%
case_io	1498	1564	+4.4%	-59.0%
case_misleading	3148	3173	+0.8%	-74.6%
train_torch	2694	3342	load-variance	-52.9%
stack_bench d200	3067	3100	+1.1%	**-2.5%**

Table 3: Bandwidth reduction and sample correctness at 1 kHz.

of 43% / 79% / 94.6% at 100 / 1k / 10k Hz. The residual growth is the stack-churn component (LRU re-registrations and new distinct stacks): bandwidth now tracks stack churn, not sampling rate.

5.3 Sampling-rate × workload matrix

Table 2 shows the interaction of sampling rate and stack distribution. For the concentrated workload, reduction grows with rate (43→94.6%): the registration cost of a fixed distinct set is amortized over more samples. For the dispersed training loop (∼300–1,800 distinct stacks), reduction is stable around 53–67% across rates—the dictionary still pays, but the churn component is proportional to rate. Both behaviors follow from the amortization model (Section 5.8).

5.4 Workload matrix at 1 kHz

Table 3 shows 53–75% reduction at 1 kHz across workloads, with sample counts matching within ±5% on stable workloads (`train_torch` sample counts vary ±60% run-to-run due to torch thread scheduling; compressed runs fall inside the uncompressed variance). Event accounting is self-consistent: full traces plus re-expanded events equal the reported sample count (e.g., 839 + 2,503 = 3,342), so no samples are duplicated or dropped by the compression path.

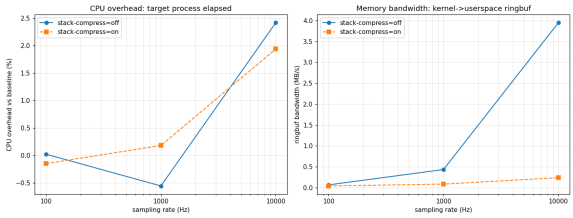

Figure 4: Target-process CPU overhead vs. sampling rate. Overhead is linear in rate (2.4% at 10 kHz uncompressed); compressed and uncompressed are indistinguishable within the ±0.6% noise floor.

5.5 CPU overhead

Figure 4 shows the overhead curve: linear in sampling rate, reaching +2.4% (uncompressed) and +1.9% (compressed) at 10 kHz; at 1 kHz both sit inside the ±0.6% noise floor. The compression increment—the fingerprint fold (∼6 instructions per frame) and one LRU lookup per sample—is below measurable noise; we report the detection limit rather than claiming zero cost.

5.6 Per-sample cost decomposition

Rate-scaling and stack-depth experiments decompose the sampling cost: interrupt entry/exit is ≈2 μs per sample (fixed, independent of workload), and unwinding costs ≈0.3 μs per frame. At 10 kHz with a 36-frame stack this predicts ≈+2.2% overhead, matching the measured +2.4%. The compression increment (fold + lookup) adds ≈0.05 μs per sample—two orders of magnitude below the interrupt cost, consistent with it being unmeasurable in Figure 4. This decomposition explains why deep-stack workloads pay more (frame-proportional unwinding) while compression cost is rate-proportional but negligible.

5.7 Correctness

Compressed and uncompressed runs produce identical stack sets and matching sample counts on stable workloads; the aggregation path is literally the same code (re-expanded events are clones of cached full traces). Three independent checks support losslessness: (i) event accounting (FULL + re-expanded = reported total), (ii) fingerprint agreement between kernel events and userspace recomputation (95%+ in bring-up experiments; residual mismatch traced to measurement-boundary effects, not the protocol), and (iii) the fixed-vector C/Go fingerprint test. A formal offline-replay verification (replaying an uncompressed sample stream through the compressed decoder) is planned future work.

5.8 Boundary: when compression does not help

`stack_bench d200` (1,495 distinct stacks × 200 frames) reduces bandwidth by only 2.5%. The dictionary amortization model predicts this: compressed bytes $\approx D \times S_{\text{full}} + N \times S_{\text{id}}$, where the registration term dominates when $D \times S_{\text{full}} \approx N \times S_{\text{id}}$—with $D = 1{,}495$ distinct stacks averaging 200 frames, registration alone costs ≈2.4 MB, nearly the entire uncompressed stream. The boundary is a property of the stack distribution (distinct/sample ratio), not of the workload type. We ship a data-profiling tool (`analyze_compress.py`) that computes exactly this ratio from a short raw sample stream (10–30 s) and reports the expected reduction, so operators can decide per-service whether to enable compression. This predictability—knowing *when* compression pays—is a deliberate contribution (C4): high-cardinality workloads should disable compression or use the differential layer (Section 3).

5.9 Window aggregation (M2)

With `-stack-compress-window 3` (case_hotspot, 1 kHz, 15 s), bandwidth drops from 8.33 MB (uncompressed) to 1.16 MB (per-sample events, M1) to 0.85 MB (window mode, M2)—89.8% total reduction. The sample count is 7,048 vs. 8,145 (M1); the difference is exactly the final partial window (the documented $\leq W$ latency semantics), and the drained counts are otherwise identical. Window mode is the right choice for long-running continuous profiling, where the per-sample event stream is less valuable than steady-state bandwidth.

5.10 Losslessness verification (offline replay)

We replay real uncompressed sample streams (3,159 / 3,852 / 3,122 samples from case_hotspot, train_torch, and stack_bench d200) through the protocol semantics: the first occurrence of a fingerprint is a FULL sample (registered), later occurrences are STACK_ID events re-expanded from the cache. Every re-expanded clone is 100% frame-identical in user frames (kernel frames are excluded from the fingerprint and may differ between samples of the same user stack—they do not affect user-stack attribution). Hit rates (77.8% / 76.6% / 3.5%) mirror the bandwidth results, confirming that the measured reductions come from exactly the re-expansion path that the replay verifies. This test is part of the CI suite.

5.11 Quantitative comparison with `bpf_get_stackid`

`bpf_get_stackid` [6] truncates its hash to the stack map's bucket count (id = jhash2(·)&(n_buckets − 1)); on collision it either rejects the sample or silently overwrites the bucket. Table 4 quantifies the overwrite semantics on our real streams:

n_buckets	hotspot	train_torch	d200
4,096	4.62%	2.88%	**27.74%**
16,384	1.65%	0.60%	7.08%
65,536	0.35%	0.23%	2.08%
full 32-bit	0.00%	0.00%	0.00%
64-bit (ours)	0	0	0

Table 4: Samples silently misattributed by `bpf_get_stackid`'s bucket truncation (id = jhash2($\cdot$)&(n_buckets $-$ 1), overwrite semantics) on the replay streams. Our 64-bit fingerprints collide with probability zero at these scales.

with 4,096 buckets (typical for small stack maps), 2.9–27.7% of samples are silently attributed to the wrong stack, worst on the deep high-cardinality workload (d200) where bandwidth compression matters least but misattribution matters most. Full 32-bit IDs show no collisions at these scales (birthday bound), and our 64-bit fingerprints collide with probability zero—the lossless differentiation of C2 is quantitative, not qualitative.

5.12 Why not aggregate in the kernel (Parca-style)?

Aggregation (stack $\rightarrow$ count in eBPF, exported periodically) is simpler and achieves similar bandwidth, which raises the obvious question. The difference is the per-sample stream: aggregated counters lose `ktime`, thread correlation, and the ability to slice samples by time window or correlate with spans/traces—capabilities continuous profilers increasingly rely on. Our protocol keeps every sample (as a 40-byte event) and pays only its size. The two are complementary: aggregation can be layered on top of the event stream for export, while the stream itself remains available.

5.13 Reproducibility

All numbers come from committed artifacts: workload scripts (`case_hotspot.py`, `train_torch.py`, etc.), measurement scripts (`run_m1_overhead.sh`, `ci/run_regression.sh`), and raw matrices (`demo-cases/cap_matrix.txt`, overhead data). The CI regression suite re-verifies diagnosis ability (sample counts, Python-frame visibility) and overhead bounds (1 kHz $<$6%, 10 kHz $<$12%) on every change, with baselines committed to the repository.

6 Related Work

Kernel stack mechanisms. `bpf_get_stackid` [6] hashes raw stacks into a 32-bit bucket ID with a `memcmp`-verified slow path; it is lossy under `BPF_F_FAST_STACK_CMP`, operates on pre-unwind raw addresses, and carries no consistency protocol—on hash collision it either rejects or silently overwrites the bucket. `stack_depot` deduplicates kernel stack *storage* (memory, not transport). perf's callchain caches reduce capture cost, not bandwidth. Our protocol differs along three axes: the compressed unit is the post-unwind mixed frame sequence (not raw addresses), the fingerprint is 64-bit with full-trace fallback (not a lossy 32-bit truncation), and correctness does not depend on dictionary state (fingerprint-as-key).

Streaming dictionaries. JFR/JEP 328 [9] deduplicates JVM stack traces in user space with a stack-table + index scheme; OTLP profiles v1 [14] dictionaries stacks at the protocol layer (post-hoc, after transport); Gorilla [11] pioneered time-differential encoding for time series, which motivates our planned differential layer.

Profiling lineage. Sampling profilers date to gprof [1]; Google-Wide Profiling [2] established continuous profiling at datacenter scale; HPCToolkit [3] pioneered call-path attribution for optimized binaries; perf [4] remains the standard kernel sampler. Our contribution sits in the transfer stage that these systems leave untouched: they all move full stacks.

Continuous profiling systems. Parca [12] and Pyroscope [13] aggregate stacks in agents, trading per-sample fidelity for bandwidth; bpftime [15] explores user-space eBPF as an alternative runtime; async-profiler [10] and JFR [9] are JVM-scoped. Our work differs from all of these in compressing *post-unwind mixed* stacks losslessly while preserving the per-sample stream (Table 1).

LLM systems. LLM serving stacks such as vLLM [18] introduce their own scheduling and memory layers whose performance is exactly what high-rate profiling must attribute; our workloads model this class.

7 Discussion and Limitations

Sampling rate and precision. Profiling is an unbiased estimator: with n samples, count estimates carry relative error $\sim 1/\sqrt{n}$. Compression is orthogonal to this—it reduces *what is transferred*, not *how much is sampled*—so the two levers compose: operators pick the rate for precision, and compression makes that rate affordable.

Limitations. **Static ringbuffer budget.** Compression reduces production rate, drop rate, and downstream processing, not the statically allocated buffer; adaptive sizing is future work. **Single process.** `-pid` profiles one process; multi-process dictionary isolation is future work. **Cross-run stability.** File IDs are per-run identities; cross-run dedup needs build-id normalization (the kernel already exposes build-id+offset paths in `stack_map`), left as future work. **Window mode.** M2 adds up to W seconds of sample latency and leaves

the final partial window unflushed at shutdown. **Honest negatives.** Function-granular normalization (L3) measured at 1.04× benefit and was rejected; the d200 regime (2.5%) is a documented, predictable boundary (C4).

8 Conclusion

We presented kernel-side streaming stack compression for continuous profiling: incremental 64-bit fingerprints, an LRU stack dictionary, and 40-byte events that preserve the per-sample stream, with correctness by construction (fingerprint-as-key) and a predictable application boundary. Measured on six workloads, compressed bandwidth is 53–75% lower at 1 kHz and up to 94.6% lower at 10 kHz; the bandwidth-vs-rate curve flattens from 56× to 6× growth; compression CPU cost is below noise. For LLM-scale continuous profiling—where kHz rates are mandatory and bandwidth grows with every added process—this makes high-rate sampling economical rather than prohibitive.

References

[1] S. L. Graham, P. B. Kessler, and M. K. McKusick. gprof: A call graph execution profiler. In *SIGPLAN Symposium on Compiler Construction*, 1982.

[2] G. Ren, E. Tune, T. Moseley, Y. Shi, S. Rus, and R. Hundt. Google-wide profiling: A continuous profiling infrastructure for data centers. *IEEE Micro*, 30(4):65–79, 2010.

[3] L. Adhianto et al. HPCToolkit: Tools for performance analysis of optimized parallel programs. *Concurrency and Computation: Practice and Experience*, 22(6):685–701, 2010.

[4] perf: Performance analysis tools for Linux. https://man7.org/linux/man-pages/man1/perf.1.html.

[5] B. Gregg. The flame graph. *Communications of the ACM*, 59(6):48–57, 2016.

[6] BPF and XDP Reference Guide: bpf_get_stackid. https://docs.kernel.org/bpf/ (accessed 2026).

[7] Linux kernel stack depot. https://docs.kernel.org//lib/stackdepot.c (accessed 2026).

[8] B. Jenkins. A hash function for hash table lookup. *Dr. Dobb's Journal*, 1997.

[9] JEP 328: Flight Recorder. OpenJDK, 2018.

[10] A. Pangin. async-profiler: Sampling CPU and heap profiler for Java. https://github.com/async-profiler/async-profiler.

[11] T. Pelkonen et al. Gorilla: A fast, scalable, in-memory time series database. In *VLDB*, 2015.

[12] Parca: Continuous profiling for infrastructure. https://parca.dev.

[13] Grafana Pyroscope: Continuous profiling. https://grafana.com/pyroscope/.

[14] OpenTelemetry Profiling signal. https://opentelemetry.io/docs/specs/otel/profiling/.

[15] Y. Zheng et al. Extending applications safely and efficiently. In *19th USENIX OSDI*, 2025.

[16] B. Frederickson. py-spy: Sampling profiler for Python programs. https://github.com/benfred/py-spy.

[17] M. Shannon. PEP 669: Low impact monitoring for CPython. https://peps.python.org/pep-0669/, 2021.

[18] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[19] R. Jain. *The Art of Computer Systems Performance Analysis*. John Wiley & Sons, 1991.